

Multi-Prompt Benchmark Report

12 models x 5 prompt styles (code, iambic pentameter, clinical prose, creative writing, math proof) | NVIDIA Jetson Orin Nano 8GB | llama.cpp 0b1bad1 | GUI off | -ngl 99 -fa on --temp 0.3

1. Generation Speed (tok/s)

Tokens generated per second during inference. Consistent across prompt types because generation is memory-bandwidth bound (reading weights), not compute bound.

Model	Params	Quant	Size	Code	iambic	Prose	Creative	Math	Avg
CodeGemma 2B	2.51B	Q4_0	1.44 GiB	30.6	30.7	30.7	30.6	30.7	30.7
Granite 3.0 2B	2.63B	Q4_K_M	1.49 GiB	27.1	27.4	27.4	27.4	27.2	27.3
Granite 3.2 2B	2.53B	Q4_K_M	1.44 GiB	27.1	27.3	27.3	27.3	27.3	27.3
Gemma 4 E2B	4.63B	Q4_0	2.63 GiB	26.7	26.8	26.7	26.8	26.7	26.7
Gemma 2 2B	2.61B	Q4_0	1.51 GiB	25.3	25.1	25.2	25.2	25.2	25.2
LFM 2.5 2.6B	2.70B	Q4_K_M	1.55 GiB	25.3	25.2	25.3	25.3	25.2	25.3
Qwen 2.5 3B	3.09B	Q4_K_M	1.79 GiB	21.5	21.4	21.5	21.5	21.5	21.5
Hermes 3 3B	3.21B	Q4_K_M	1.87 GiB	20.8	20.8	20.8	20.8	20.8	20.8
Llama 3.2 3B	3.21B	Q4_K_M	1.87 GiB	20.8	20.6	20.8	20.8	20.8	20.8
Granite 4 3B	3.40B	Q4_K_M	1.95 GiB	19.6	19.7	19.7	19.7	19.7	19.7
Phi-3 3.8B	3.82B	Q4_0	2.03 GiB	20.7	21.2	20.7	20.7	20.8	20.8
SmallThinker 3B	3.40B	Q8_0	3.36 GiB	19.2	19.2	19.3	19.2	19.2	19.2

Highlight: CodeGemma 2B leads at 30.7 tok/s avg. The 3B class (Qwen, Hermes, Llama, Phi) clusters tightly at 20-22 tok/s. Gemma 4 E2B (MatFormer) and LFM 2.5 bridge the gap at 25-27 tok/s.

2. Prompt Eval Speed (tok/s) — Context Ingestion

How fast the model processes the input prompt. Varies by prompt type due to token distribution and attention patterns. Higher = faster time-to-first-token.

Model	Code	iambic	Prose	Creative	Math	Avg
CodeGemma 2B	438	438	532	589	413	482
Granite 3.0 2B	250	337	348	265	300	300
Granite 3.2 2B	430	525	456	486	481	476
Gemma 4 E2B	128	125	141	142	104	128
Gemma 2 2B	235	236	263	256	218	242
LFM 2.5 2.6B	233	250	282	282	183	246
Qwen 2.5 3B	380	346	397	421	352	379
Hermes 3 3B	295	302	410	362	366	347
Llama 3.2 3B	378	429	465	474	385	426
Granite 4 3B	329	335	334	341	313	330
Phi-3 3.8B	329	303	415	448	299	359
SmallThinker 3B	289	367	299	310	269	307

Highlight: CodeGemma leads at 482 tok/s avg. Gemma 4 E2B is slowest at 128 tok/s due to thinking model overhead (processes internal reasoning context). Creative/prose prompts generally process faster than code/math across all models.

3. Tokens Generated — Output Completeness

Word count of generated output. Lower counts may indicate truncated or incomplete responses. Higher counts suggest more thorough answers (but not necessarily better quality).

Model	Code	Iambic	Prose	Creative	Math	Avg
CodeGemma 2B	185	40	207	283	74	158
Granite 3.0 2B	257	181	127	131	306	200
Granite 3.2 2B	215	159	127	245	278	205
Gemma 4 E2B	256	130	197	221	193	199
Gemma 2 2B	186	61	268	281	256	210
LFM 2.5 2.6B	275	168	180	270	200	219
Qwen 2.5 3B	219	54	255	291	274	219
Hermes 3 3B	175	66	262	278	269	210
Llama 3.2 3B	243	47	250	274	238	210
Granite 4 3B	246	136	248	354	316	260
Phi-3 3.8B	141	126	176	235	222	180
SmallThinker 3B	49	49	49	49	49	49

Highlight: LFM 2.5 and Qwen 2.5 produce the most output (219 avg). CodeGemma produces the least (158 avg) and failed on iambic pentameter (40 tokens of chat template tokens). Gemma 4 E2B and LFM 2.5 are thinking models that spend tokens on internal reasoning.

4. Quality Observations by Prompt Type

Prompt Type	Best Model	Runner-Up	Key Observation
Code Generation	codegemma:2b	granite3-dense:2b	CodeGemma fastest + correct; Granite good general code
Iambic Pentameter	gemma2:2b	granite3-dense:2b	Gemma2 best meter (10-syllable); Granite good form
Clinical Prose	qwen2.5:3b	gemma2:2b	Qwen most detailed; Gemma2 best structured; all adequate
Creative Writing	qwen2.5:3b	lfm2.5:2.6b	Qwen most vivid; LFM good narrative flow
Math Proof	qwen2.5:3b	gemma2:2b	Qwen cleanest proof; Gemma2 proper theorem format

5. Summary Recommendations

Use Case	Best Model	Gen tok/s	Why
Fastest overall	codegemma:2b	30.7	Fastest gen + prompt eval, but code-only quality
Best 2B generalist	granite3-dense:2b	27.3	Consistent quality, fast, good code+prose
All-rounder (3B)	qwen2.5:3b	21.5	Consistent quality across all 5 prompt styles
Code generation	codegemma:2b	30.7	Correct output, code specialist architecture
Creative/prose	gemma2:2b	25.2	Best poetry meter, strong clinical prose
Math/proofs	qwen2.5:3b	21.5	Cleanest proof structure, correct reasoning
Reasoning/thinking	gemma4 E2B	26.7	Thinking model with reasoning tokens
Most complete output	lfm2.5:2.6b	25.3	Highest avg token count across prompts

6. Output Samples

Representative output previews for quality comparison. Truncated to ~400 chars.

Iambic Pentameter — gemma2:2b (best)

The leaves descend, a fiery, vibrant show,
As summer's warmth gives way to chilling breeze.
The harvest's bounty, now all brown and low,
Whispers of winter's icy, silent ease.

The sun descends, its golden rays now faint,
And shadows grow, as nature's colors fade.
The air grows crisp, a frosty, silent paint,
A blanket of white, upon the land is laid.

Iambic Pentameter — codegemma:2b (failed - chat loop)

user

assistant

user

assistant

user

assistant

user

assistant

user

assistant

user

assistant

user

assistant

user

assistant

<

Math Proof — qwen2.5:3b (best)

Theorem: $\sqrt{2}$ is irrational.

Proof by contradiction:

1. Assume that $\sqrt{2}$ is a rational number. By definition, a rational number can be expressed as a fraction a/b where a and b are integers with no common factors other than 1 (i.e., the fraction is in its simplest form) and b is not zero.
2. Therefore, we can write $\sqrt{2} = a/b$, where a and b are integers with no common factors other than 1, and $b \neq 0$...

Math Proof — gemma2:2b (runner-up)

Theorem: $\sqrt{2}$ is irrational.

Proof by Contradiction:

1. **Assumption:** Assume that $\sqrt{2}$ is rational.
- This means we can express $\sqrt{2}$ as a fraction in its simplest form, where the numerator and denominator are integers with no common factors other than 1. Let's represent this fraction as $\sqrt{2} = a/b$, where a and b are integers and $b \neq 0$.
2. **Square both sides:** Squaring both sides of the ...

Code — codegemma:2b (specialist)

Sure, here's an implementation of the `merge\_sort` function in Python:

```
```python
def merge_sort(lst: list) -> list:
 """
 Sorts a list of integers in ascending order using the merge sort algorithm.

 Args:
 lst (list): The list to be sorted.
 """
```

```
Returns:
list: A new list containing the integers in ascending order.
```

```
Raises:
TypeError: If `lst` is not a list.
...
```

## Creative — qwen2.5:3b (best)

The lighthouse stood like a solitary sentinel in the storm, its spire a dark silhouette against the howling tempest. Inside, the keeper, an old woman with weathered skin and eyes that held stories of solitude, sat by the fire, her hands wrapped around a mug of steaming tea. The room was dim, the only light from the flickering flames and the occasional flash of lightning outside.

The door creaked...